

Adding Interactivity in React: Hinglish Explanation

Is chapter ka main focus hai ki **React components ko interactive kaise banaya jata hai**. Interactive ka matlab hai ki user ke action ke response mein screen par kuch change ho, jaise:

- Button click karne par counter badhna
- Image gallery mein next image dikhna
- Form mein type karte hi value update hona
- “Show details” par extra information dikhna
- Checkbox select karne par item ka status change hona

React mein time ke saath change hone wale data ko **state** kehte hain.

1. Responding to Events

User jab kisi element ke saath interact karta hai, tab browser ek event generate karta hai.

Common events:

- onClick: click hone par
- onChange: input ki value change hone par
- onSubmit: form submit hone par
- Focus, hover aur keyboard events bhi ho sakte hain

React mein events handle karne ke liye hum **event handler function** banate hain.

```
1  function handleClick() {  
2    alert('Button clicked!');  
3  }  
4  
5  <button onClick={handleClick}>  
6    Click me  
7  </button>
```

Yahan:

1. User button click karta hai.
2. onClick event trigger hota hai.
3. handleClick function execute hota hai.
4. Alert show hota hai.

Important difference

Function ko pass karna hai:

```
1 <button onClick={handleClick}>
```

Function ko turant call nahi karna:

```
1 <button onClick={handleClick()}>
```

handleClick() likhne par function rendering ke time hi execute ho jayega, click ke time nahi.

Custom components aur event props

Built-in HTML component jaise <button> browser ke standard events support karta hai, jaise onClick.

Lekin apne custom component ke event prop ka naam hum application ki requirement ke according rakh sakte hain:

```
1 <Toolbar
2   onPlayMovie={() => alert('Playing!')}
3   onUploadImage={() => alert('Uploading!')}
4 />
```

Toolbar component in functions ko props ke through receive karta hai:

```
1 function Toolbar({ onPlayMovie, onUploadImage }) {
2   return (
3     <div>
4       <Button onClick={onPlayMovie}>
5         Play Movie
6       </Button>
7
8       <Button onClick={onUploadImage}>
9         Upload Image
10      </Button>
11    </div>
12  );
13 }
```

Flow kuch aisa hai:

```
1 App
2   └─ Toolbar
3       └─ onPlayMovie
4       └─ onUploadImage
5           ↓
6         Button
7           ↓
```

8 HTML button

Custom event prop ka naam application-specific ho sakta hai, jaise:

```
1  onPlayMovie
2  onUploadImage
3  onSave
4  onDelete
5  onNext
```

Lekin final built-in <button> ko browser ka actual event onClick hi diya jata hai.

2. State: Component ki Memory

Components ko kai baar kuch information remember karni padti hai.

Examples:

- Current image ka index
- Input mein likha hua text
- Details visible hain ya hidden
- Shopping cart ke products
- Checkbox checked hai ya nahi

React mein component ki is memory ko **state** kaha jata hai.

State create karne ke liye useState Hook use hota hai:

```
1  import { useState } from 'react';
```

Basic syntax:

```
1  const [stateValue, setStateValue] = useState(initialValue);
```

Example:

```
1  const [index, setIndex] = useState(0);
```

Yahan:

- index: current state value
- setIndex: state ko update karne wala function
- 0: initial value

Boolean state ka example:

```
1  const [showMore, setShowMore] = useState(false);
```

Initially showMore ki value false hai.

Image gallery ka state

Gallery mein do state variables hain:

```
1  const [index, setIndex] = useState(0);
2  const [showMore, setShowMore] = useState(false);
```

index

Current sculpture ka position remember karta hai.

```
1  let sculpture = sculptureList[index];
```

Agar index 0 hai, array ka first sculpture milega.

Agar index 1 hai, second sculpture milega.

Next button ka logic

```
1  const hasNext = index < sculptureList.length - 1;
```

Ye check karta hai ki current sculpture ke baad koi aur sculpture available hai ya nahi.

```
1  function handleNextClick() {
2    if (hasNext) {
3      setIndex(index + 1);
4    } else {
5      setIndex(0);
6    }
7  }
```

Logic:

- Agar next sculpture available hai, index mein 1 add karo.
- Agar current sculpture last hai, index ko wapas 0 kar do.

Isse gallery circular ban jati hai:

```
1  0 → 1 → 2 → ... → last → 0
```

Show/Hide details

```
1  function handleMoreClick() {
2    setShowMore(!showMore);
3  }
```

! boolean ko reverse karta hai:

```
1  false → true
```

```
2 true → false
```

Button ka text bhi condition ke according change hota hai:

```
1 {showMore ? 'Hide' : 'Show'} details
```

Ye ternary operator hai:

```
1 condition ? valueIfTrue : valueIfFalse
```

Description conditionally render hoti hai:

```
1 {showMore && <p>{sculpture.description}</p>}
```

Agar showMore true hai, paragraph render hoga. False hone par paragraph nahi dikhega.

3. Render and Commit

Screen par component dikhne se pehle React UI update process ko kuch phases mein complete karta hai.

Chapter mein is process ke teen steps diye gaye hain:

1. **Trigger**
 2. **Render**
 3. **Commit**
-

Step 1: Triggering a render

Render trigger ho sakta hai:

- Component initially display hone par
- State update hone par

For example:

```
1 setIndex(index + 1);
```

Ye React se kehta hai:

State ko update karke component ko dobara render karo.

Step 2: Rendering the component

React component function ko dobara execute karta hai:

```
1 function Gallery() {  
2   // Component code
```

```
3 }
```

Component updated state ke according naya JSX calculate karta hai.

Important point:

Rendering ka matlab component function ko call karke naya UI description calculate karna hai.

Is phase mein React determine karta hai ki screen par kya dikhna chahiye.

Step 3: Committing to the DOM

React calculated output ko existing DOM se compare karta hai aur required changes DOM mein apply karta hai.

For example, agar sirf sculpture ka title aur image change hui hai, toh React relevant DOM parts ko update karega.

Simple flow:

```
1  User interaction
2      ↓
3  State update
4      ↓
5  Render trigger
6      ↓
7  Component executes
8      ↓
9  New JSX calculate hota hai
10     ↓
11  React DOM mein changes commit karta hai
```

4. State as a Snapshot

React state normal JavaScript variable ki tarah instantly mutate nahi hoti.

Example:

```
1  console.log(count); // 0
2  setCount(count + 1);
3  console.log(count); // Still 0
```

Aisa isliye hota hai kyunki setCount() currently running code ke andar count ko change nahi karta.

Ye next render ke liye state update request karta hai.

Current event handler ke andar count ki value ek **snapshot** ki tarah fixed hoti hai.

```
1 Current render snapshot: count = 0
2 setCount(1): next render ki request
3 Current code: count abhi bhi 0
4 Next render: count = 1
```

Chat form example

Form mein state hai:

```
1 const [to, setTo] = useState('Alice');
2 const [message, setMessage] = useState('Hello');
```

Submit hone par:

```
1 function handleSubmit(e) {
2   e.preventDefault();
3
4   setTimeout(() => {
5     alert(`You said ${message} to ${to}`);
6   }, 5000);
7 }
```

Agar send karte waqt:

```
1 to = Alice
2 message = Hello
```

To timeout wala function us render ke to aur message values ko remember karta hai.

Agar send karne ke baad recipient ko Bob kar diya jaye, toh pending alert mein purane render ka snapshot use hoga.

Result:

```
1 You said Hello to Alice
```

Is concept ka benefit ye hai ki ek particular render ke event handlers ke andar state consistent rehti hai.

5. Queueing Multiple State Updates

Is example mein +3 button teen baar increment() call karta hai:

```
1 function increment() {  
2   setScore(score + 1);  
3 }
```

```
1 increment();  
2 increment();  
3 increment();
```

Expected result ho sakta hai 3, lekin score sirf 1 badhta hai.

Kyun?

Maan lo current render mein:

```
1 score = 0
```

Teen calls effectively ye karti hain:

```
1 setScore(0 + 1);  
2 setScore(0 + 1);  
3 setScore(0 + 1);
```

Har call current render ke same score snapshot ko use kar rahi hai.

Teeno 1 set karne ki request karti hain:

```
1 setScore(1)  
2 setScore(1)  
3 setScore(1)
```

Isliye final result 1 hota hai, 3 nahi.

Updater function se fix

Correct solution:

```
1 function increment() {  
2   setScore(s => s + 1);  
3 }
```

Yahan setter ko direct value ke badle **updater function** diya gaya hai.

React queued updater functions ko sequence mein process karta hai:

```
1 Initial score = 0  
2  
3 s => s + 1      0 → 1  
4 s => s + 1      1 → 2  
5 s => s + 1      2 → 3
```

Final result:

```
1 Score: 3
```

Kab updater function use karna chahiye?

Jab next state previous state par depend karti ho:

```
1 setCount(c => c + 1);
2 setScore(s => s + 1);
3 setItems(items => [...items, newItem]);
```

Simple rule:

```
1 Previous state se next state calculate ho rahi hai
2 → updater function use karo
```

6. Updating Objects in State

React state mein objects store kiye ja sakte hain:

```
1 const [person, setPerson] = useState({
2   name: 'Niki de Saint Phalle',
3   artwork: {
4     title: 'Blue Nana',
5     city: 'Hamburg'
6   }
7 });
```

Lekin state object ko directly modify nahi karna chahiye.

Wrong approach:

```
1 person.name = 'New Name';
```

Aisa karna existing object ko mutate karta hai.

Correct approach hai ek **naya object** banana:

```
1 setPerson({
2   ...person,
3   name: 'New Name'
4 });
```

Spread syntax

```
1 ...person
```

Existing person object ke properties ko naye object mein copy karta hai.

```
1  setPerson({
2    ...person,
3    name: e.target.value
4  });
```

Conceptually:

```
1  {
2    name: person.name,
3    artwork: person.artwork,
4    name: e.target.value
5  }
```

Last name property previous name ko override kar deti hai.

Nested object update

artwork object, person ke andar nested hai:

```
1  person.artwork.title
```

Nested value update karne ke liye har affected level ko copy karna padta hai:

```
1  setPerson({
2    ...person,
3    artwork: {
4      ...person.artwork,
5      title: e.target.value
6    }
7  });
```

Ismein:

1. Outer person copy hota hai.
2. Inner artwork copy hota hai.
3. title ki new value set hoti hai.

Result ka structure:

```
1  {
2    ...oldPerson,
3    artwork: {
4      ...oldArtwork,
5      title: newTitle
6    }
7  }
```

Agar sirf outer object copy karke inner object directly modify karoge, toh nested object ka original reference mutate ho sakta hai.

7. Immer se Object Updates

Nested objects ko repeatedly spread karna verbose ho sakta hai:

```
1  setPerson({
2    ...person,
3    artwork: {
4      ...person.artwork,
5      title: e.target.value
6    }
7  });
```

Provided chapter mein **Immer** library ka alternative diya gaya hai.

```
1  import { useImmer } from 'use-immers';
1  const [person, updatePerson] = useImmer(initialPerson);
```

Update syntax:

```
1  updatePerson(draft => {
2    draft.artwork.title = e.target.value;
3  });
```

Ye code direct mutation jaisa dikhta hai, lekin draft ke through update kiya ja raha hai.

Isse nested state update ka code compact ho jata hai:

```
1  updatePerson(draft => {
2    draft.name = e.target.value;
3  });

1  updatePerson(draft => {
2    draft.artwork.city = e.target.value;
3  });
```

Chapter mein external library ke roop mein diya gaya link:

- <https://github.com/immerjs/use-immers>
-

8. Updating Arrays in State

JavaScript arrays bhi mutable objects hain. Isliye React state mein stored array ko bhi read-only treat karna chahiye.

Example state:

```
1 const [list, setList] = useState(initialList);
```

Wrong approach:

```
1 list[0].seen = true;
2 setList(list);
```

Ye existing array/object ko directly mutate karta hai.

Correct approach: ek naya array create karo.

map() se array item update karna

```
1 function handleToggle(artworkId, nextSeen) {
2   setList(
3     list.map(artwork => {
4       if (artwork.id === artworkId) {
5         return {
6           ...artwork,
7           seen: nextSeen
8         };
9       } else {
10        return artwork;
11      }
12    })
13  );
14 }
```

map() har element ko process karta hai aur ek naya array return karta hai.

Flow:

```
1 Har artwork ko check karo
2   ↓
3 Kya ID target ID ke equal hai?
4   ├── Yes → Object ki copy + updated seen
5   └── No  → Existing artwork return
6   ↓
7 Naya array create
```

```
8      ↓  
9  setList(newArray)
```

Target item ke liye:

```
1  return {  
2    ...artwork,  
3    seen: nextSeen  
4  };
```

Baaki items unchanged return hote hain:

```
1  return artwork;
```

Checkbox interaction

```
1  <input  
2    type="checkbox"  
3    checked={artwork.seen}  
4    onChange={e => {  
5      onToggle(  
6        artwork.id,  
7        e.target.checked  
8      )};  
9    }}  
10 />
```

Yahan:

- checked={artwork.seen} checkbox ko state ke saath connect karta hai.
- e.target.checked new boolean value deta hai.
- artwork.id batata hai ki kaunsa item update karna hai.
- onToggle() parent component ke handler ko call karta hai.

Flow:

```
1  Checkbox changed  
2    ↓  
3  onChange event  
4    ↓  
5  artwork.id + checked value  
6    ↓  
7  handleToggle()  
8    ↓  
9  map() creates new array  
10   ↓
```

```
11 setList()  
12   ↓  
13 Component re-render
```

Arrays ke liye Immer

Immer ke saath:

```
1 const [list, updateList] = useImmer(initialList);
```

Item find karo:

```
1 const artwork = draft.find(  
2   a => a.id === artworkId  
3 );
```

Update karo:

```
1 artwork.seen = nextSeen;
```

Complete handler:

```
1 function handleToggle(artworkId, nextSeen) {  
2   updateList(draft => {  
3     const artwork = draft.find(  
4       a => a.id === artworkId  
5     );  
6  
7     artwork.seen = nextSeen;  
8   });  
9 }
```

Ye normal mutation jaisa readable lagta hai, lekin update draft ke through hota hai.

Objects aur Arrays ko Directly Mutate Kyun Nahi Karna?

React state ko read-only treat karne ke important practical reasons hain:

- React ko updated value ka naya reference milta hai.
- Previous state accidentally change nahi hoti.
- State updates predictable rehte hain.
- Purane render ka snapshot stable rehta hai.
- Debugging comparatively easier hoti hai.

Reference difference:

```
1  const oldPerson = person;
2
3  const newPerson = {
4    ...person,
5    name: 'New name'
6  };
7
8  oldPerson !== newPerson;
```

Array ke saath:

```
1  const newList = list.map(...);
2
3  list !== newList;
```

React ko ek new object ya array diya jata hai.

Complete Mental Model

Is poore chapter ko ek flow se samjho:

```
1  1. User interaction karta hai
2      ↓
3  2. Event handler execute hota hai
4      ↓
5  3. State setter call hota hai
6      ↓
7  4. React render schedule karta hai
8      ↓
9  5. Component updated state ke saath run hota hai
10     ↓
11  6. Naya JSX calculate hota hai
12     ↓
13  7. React required DOM changes commit karta hai
14     ↓
15  8. Screen updated output dikhati hai
```

Important Rules to Remember

Rule 1: Event handler ko pass karo

```
1 onClick={handleClick}
```

Generally event prop mein immediately function call mat karo:

```
1 onClick={handleClick()}
```

Rule 2: State ko setter function se update karo

```
1 setCount(count + 1);
```

Direct assignment nahi:

```
1 count = count + 1;
```

Rule 3: State current render ka snapshot hoti hai

```
1 setCount(count + 1);  
2 console.log(count);
```

console.log current render ki purani value hi dekhenga.

Rule 4: Previous state par dependent update mein updater use karo

```
1 setCount(c => c + 1);
```

Especially jab multiple updates queue karni hon.

Rule 5: Objects ko copy karke update karo

```
1 setPerson({  
2   ...person,  
3   name: newName  
4 });
```

Rule 6: Nested object ke har affected level ko copy karo

```
1 setPerson({  
2   ...person,  
3   artwork: {  
4     ...person.artwork,
```

```
5     title: newTitle
6   }
7 });
```

Rule 7: Array update ke liye new array banao

Item modify karne ke liye:

```
1  setList(
2    list.map(item =>
3      item.id === targetId
4        ? { ...item, seen: true }
5        : item
6    )
7  );
```

Quick Revision

- **Event handler:** user interaction ke response mein chalne wala function.
- **State:** component ki memory.
- **useState:** state variable declare karne ka Hook.
- **Setter function:** state update request karta hai.
- **Render:** component function se UI calculate hoti hai.
- **Commit:** calculated changes DOM mein apply hote hain.
- **Snapshot:** har render ko state ka fixed version milta hai.
- **Updater function:** previous state se next state calculate karta hai.
- **Spread syntax:** object ya array copy karne mein useful hai.
- **Immer:** complex nested updates ko concise banane ka option hai.

One-line summary

React interactivity ka core formula hai: user event → event handler → state update → re-render → updated UI.